Campus FEB

Documentação técnica

Arquitetura, API, banco de dados, segurança, testes e deploy
Versão de 30/10/2026 · Hackathon XXIII SESINFO — UNIFEB

Produção: https://campusfeb.roqueemanuel2018.workers.dev

1. Visão da arquitetura

Um único **Cloudflare Worker** serve a API em `/api/*` e o front-end React compilado (Workers Static Assets) na mesma origem. Assim não há CORS, o deploy é um comando só e tudo roda na borda da Cloudflare, perto do usuário.

Camada	Tecnologia	Papel
Front-end	React 18, TypeScript, Vite 7, Tailwind CSS v4, React Router 7	Interface mobile first, instalável como PWA
Mapas	Leaflet 1.9 (Mapa e Blocos), MapLibre GL 6 (3D, carregado sob demanda)	Três vistas do campus
API	Hono no Cloudflare Workers, validação com Zod	Regras de acesso, validação e respostas claras
Regras de negócio	`packages/core` (TypeScript puro)	QR, distância, presença, senhas, grupos, busca, geometria do mapa
Dados	Cloudflare D1 (SQLite)	Usuários, eventos, presenças, locais e conteúdo
Arquivos	Cloudflare KV (`campusfeb-photos`)	Fotos dos locais (original e miniatura)
Agendamento	Cron Trigger do Worker	Cópia dos banners do site do UNIFEB a cada 6 horas

> As regras de negócio não importam React, Hono nem APIs da Cloudflare (arquitetura limpa). Isso deixa o núcleo testável com Vitest e permite trocar o framework sem reescrever as regras.

2. Estrutura do repositório

Pasta	Conteúdo
`apps/web`	React: páginas do aluno (`student/`), coordenador (`coord/`), Atlética (`org/`), admin (`admin/`), componentes de mapa, PWA (`public/`)
`apps/web/scripts`	`fetch-campus-osm.mjs` : baixa os prédios do OpenStreetMap e gera `src/data/campus-osm.json`
`apps/api/src`	`app.ts` (rotas), `routes/` , `lib/` (sessão, erros HTTP), `services/`
`apps/api/migrations`	4 migrações SQL do D1 (`0001` a `0004`)
`apps/api/scripts`	Seed com dados de demonstração (apaga tudo: só para ambiente local)
`packages/core`	Regras puras e testadas (`campus.ts` , `campus-map.ts` , `photos.ts` , QR, presença, senhas)

`openspec/`	Propostas, especificações e tarefas (fluxo OpenSpec)
`docs/`	Perguntas e decisões, estes PDFs

3. API

Todas as rotas ficam sob `/api`, recebem e devolvem JSON (exceto upload de fotos, que é multipart) e respondem erros no formato `{ error, message, field? }` com mensagens em português.

Grupo	Rotas principais	Acesso
`/auth`	POST login, register, logout · GET me	Público / logado
`/courses`	Lista de cursos (cadastro)	Público
`/events`	GET lista e detalhe · POST/DELETE `:id/register` (inscrição)	Aluno
`/me`	GET events (meus eventos) · GET hours (horas complementares)	Aluno
`/scan`, `/checkin`, `/events/:id/keyword`	Leitura do QR, confirmação de presença com local e aparelho, palavra-chave de saída	Aluno
`/places`	GET lista (com fotos) · POST, PUT, DELETE	Público / admin
`/place-photos`	POST envio (multipart) · DELETE · GET `:id/full|thumb` com cache longo	Admin / público
`/groups`	Lista, detalhe, criar, entrar e sair (`:id/join`), excluir	Aluno
`/announcements`	Lista por público, marcar lido, publicar, excluir	Aluno / coordenador / admin
`/resources`	Materiais: listar, compartilhar, excluir (dono ou admin)	Aluno
`/accessibility-reports`	Relatar, listar, `:id/resolve`	Aluno / admin
`/social-events`	Jogos da Atlética: CRUD, `:id/rsvp`, `:id/rsvps`	Público / Atlética
`/banners`	GET banners · POST sync (forçar cópia)	Público / admin
`/coord`	Eventos, fases, QR, presenças, revisão, lançamento manual, tentativas, lista impressa	Coordenador
`/admin`	Cursos, coordenadores, contas da Atlética, rede da faculdade (IP do Wi-Fi)	Admin

4. Banco de dados (D1)

Área	Tabelas
Contas e acesso	`users`, `courses`, `sessions`, `login_attempts`, `settings`
Palestras e presença	`events`, `registrations`, `attendance`, `scan_tickets`, `checkin_attempts`
Mapa	`places`, `place_photos` (arquivos no KV: `place-photo:<id>:full|thumb`)
Vida universitária	`study_groups`, `study_group_members`, `announcements`, `announcement_reads`, `resources`, `accessibility_reports`
Atlética e site	`social_events`, `social_event_rsvps`, `site_banners`

Migrações: 0001 (base do SyncFEB), 0002 (campus), 0003 (Atlética, vagas e banners; recria `users` com o papel `organizer`), 0004 (fotos dos locais).

5. Check-in por QR Code

1. O coordenador abre o **Telão**: o QR é assinado pelo servidor e troca a cada **10 segundos** (aceita até 5 s de atraso da câmera e da rede).
2. O aluno lê o QR no app; o servidor gera um ticket de leitura de uso único.
3. Na confirmação, o servidor confere: **local** (GPS dentro do raio do evento, com precisão até 150 m, ou IP do Wi-Fi da faculdade), **aparelho** (um aparelho por aluno no evento) e a **fase** (entrada ou saída).
4. QR vencido ou aparelho repetido: bloqueia. Fora do raio: presença **suspeita**, que o coordenador confirma ou recusa.
5. Modos por evento: só entrada, só saída, entrada + saída, ou entrada + palavra-chave no fim (até 3 tentativas).
6. Presença concluída soma as horas do evento em `/me/hours`. Sem celular: lista impressa e lançamento manual.

6. Mapa do campus

- **Mapa**: Leaflet com satélite (Esri) ou ruas (OpenStreetMap), pinos por categoria, busca sem acento e filtros.
- **Blocos**: desenho dos 47 prédios do OpenStreetMap, sem imagens. `layoutCampus` (core) dá nome a cada prédio pelos locais dentro dele (ou a até 15 m), junta prédios do mesmo bloco e calcula os andares.
- **3D**: MapLibre GL com prédios extrudados (3,6 m por andar), carregado só quando escolhido; mostra aviso se o aparelho não tiver WebGL.
- **Fotos**: o navegador reduz antes de enviar (1600 px e miniatura de 480 px); o servidor confere o tipo pelos bytes (JPEG, PNG, WebP), limita a 2 MB e 6 fotos por local.
- Seleção e vista ficam na URL (`?vista=blocos&local=...`), então dá para compartilhar o link de um local.

7. Segurança

- Senhas com **PBKDF2** (Web Crypto) e sal por usuário.
- Sessão em cookie HttpOnly com validade de 7 dias, guardada no D1 (pode ser revogada).
- Login bloqueado por 15 minutos após 5 tentativas erradas; mesma mensagem para RA inexistente e senha errada.
- Toda entrada validada com Zod; permissões por papel em cada rota (`requireRole`).
- Segredos só em `wrangler secret`; nada de senha no código de produção.
- Upload só para admin, com checagem de tipo pelos bytes e `X-Content-Type-Options: nosniff`.

8. Testes

139 testes com Vitest: as regras de `packages/core` (QR, distância, presença, busca, agrupamento dos blocos, fotos) e a API inteira contra um SQLite em memória (login e bloqueio, check-in e suas travas, mapa e fotos, grupos, avisos e vagas, materiais, Atlética, banners e relatos).

Comando: `npm test` · Tipos: `npm run typecheck`

9. Rodar localmente

1. Node 22.13+ e `npm install`
2. `npm run db:migrate:local` e `npm run db:seed:local`
3. `npm run build` e `npm run dev -w apps/api` → http://localhost:8788
4. Para editar o front com recarga: `npm run dev -w apps/web` → http://localhost:5173

10. Deploy

- `npm run deploy`: compila o React e publica o Worker `campusfeb` (D1 `campusfeb`, KV `campusfeb-photos`).
- Migrações em produção: `npm run db:migrate:remote -w apps/api`.
- **Nunca** rodar o seed em produção: ele apaga todos os dados.
- Separado do projeto 1 (SyncFEB), que continua publicado no próprio Worker.

11. Decisões e limites conhecidos

- Fotos no KV porque o R2 não está ativado na conta; o volume (até 6 fotos de ~300 KB por local) cabe bem.
- Nomes de salas são exemplos; posições e desenho vêm do OpenStreetMap. Correção pelo Admin → Mapa.
- A altura dos prédios no 3D é estimada pelos andares informados nos locais.
- IA de certificados ficou fora da demo para não gerar custo (próximo passo).